

Adobe ColdFusion RCE — SOC Investigation Report

1. Executive Overview

This investigation analyzes a security incident involving an **Adobe ColdFusion web server** that was compromised and subsequently used for command execution, reconnaissance, and remote communication.

The investigation began after an endpoint security system detected suspicious execution of the Windows utility:

```
nltest.exe /domain_trusts
```

The activity was unusual because the process was executed under the NT AUTHORITY\SYSTEM account and originated from the ColdFusion installation environment:

```
C:\Fusion21\cfusion\bin\
```

Analysis of the Windows Event Logs revealed that the suspicious nltest.exe execution was only one component of a larger attack sequence.

Earlier events showed encoded PowerShell commands, use of certutil.exe to decode an encoded payload, and the creation of a .cfm file inside the ColdFusion web directory:

```
C:\Fusion21\cfusion\wwwroot\cf_scripts\cfclient\huqVgdoFd.cfm
```

The decoded file contained ColdFusion Markup Language (CFML) and a cfexecute statement capable of executing operating-system commands.

Further investigation showed that the attacker used the compromised ColdFusion environment to execute:

```
net user /domain
```

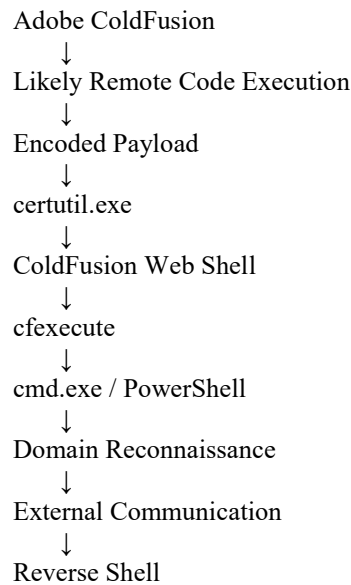
for domain reconnaissance. The resulting information was transmitted to an external webhook.site endpoint.

Additional PowerShell activity revealed an obfuscated TCP reverse-shell payload connecting to:

```
185.100.233.201:80
```

Based on the observed application, payload, and execution behavior, the challenge identifies **CVE-2023-26360** as the likely vulnerability associated with the ColdFusion compromise.

The investigation therefore reconstructed the following high-level attack chain:



The objective of this report is to document the investigation chronologically, explain the evidence that led to each finding, and identify the relevant indicators and attacker techniques.

2. Investigation Objective

The primary objective was to analyze the supplied Windows Event Log and determine:

- What caused the suspicious activity?
- How was the ColdFusion server being abused?
- What files and processes were created?
- What commands were executed?
- What external infrastructure was contacted?
- What reconnaissance was performed?
- How was the reverse shell established?
- What vulnerability was likely associated with the compromise?

The investigation was performed by correlating **process execution, command-line arguments, working directories, user context, DNS activity, and decoded payloads**.

Rather than analyzing individual events independently, the investigation followed the activity backward and forward through the process chain.

3. Evidence Source

The primary evidence source provided for the investigation was:

Event Logs.evtx

The log contains Windows security telemetry, including Sysmon events that provide information about process execution and network-related activity.

The most relevant telemetry observed during the investigation included:

- **Sysmon Event ID 1 — Process Creation**
- **Sysmon Event ID 22 — DNS Query**

Event ID 1 was particularly useful because it provided:

- Process image
- Command line
- Parent process
- Current working directory
- User context
- Execution timestamps

Event ID 22 provided DNS-resolution information that helped correlate the compromised host with external infrastructure.

4. Tools Used

4.1 Windows Event Log Analysis

The Windows Event Log was the primary source of evidence.

The investigation focused on identifying suspicious processes and then examining their:

Command Line
Parent Process
Current Directory
User
Timestamp

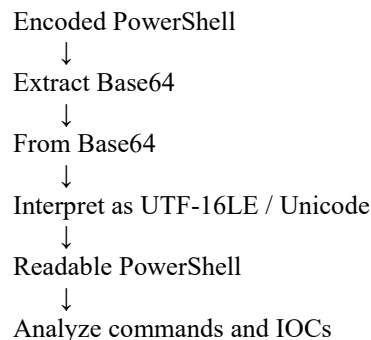
This allowed the process tree to be reconstructed.

4.2 CyberChef

CyberChef was the main tool used to decode the encoded PowerShell and ColdFusion payloads.

Several commands in the event logs were encoded using Base64. Reading the raw command line alone therefore did not reveal the actual behavior.

The general decoding workflow was:



CyberChef was also used to decode the payload that ultimately produced the malicious .cfm web shell.

This was an important part of the investigation because the encoded commands contained the actual reconnaissance and reverse-shell logic.

5. Investigation Methodology

The investigation followed a chronological and correlation-based approach.

First, the suspicious nltest.exe process was examined.

The working directory and parent process indicated that it was connected to the ColdFusion environment and PowerShell.

The investigation then moved backward through earlier PowerShell and command-line events.

This revealed:

1. External DNS activity.
2. Encoded PowerShell.
3. Domain enumeration.
4. certutil.exe decoding.
5. Creation of a ColdFusion web shell.
6. cfexecute command execution.
7. Reverse-shell functionality.

The investigation therefore developed from a single suspicious process into a complete attack chain.

6. Finding 1 — Suspicious `nltest.exe` Execution

The first significant artifact was the execution of:

`C:\Windows\System32\nltest.exe`

with the command:

`"C:\Windows\system32\nltest.exe" /domain_trusts`

The process was executing from:

`C:\Fusion21\cfusion\bin\`

and under:

`NT AUTHORITY\SYSTEM`

The parent process was PowerShell.

The execution can therefore be represented as:

```
PowerShell
  ↓
nltest.exe
  ↓
/domain_trusts
```

`nltest.exe` is a legitimate Windows utility used for domain-related operations. The important aspect of this event is therefore not simply the executable itself.

The suspicious context came from the combination of:

- `SYSTEM`-level execution
- ColdFusion working directory
- PowerShell parent process
- Domain-trust discovery command

The command:

`nltest.exe /domain_trusts`

can provide information about domain trust relationships.

For an attacker who has already obtained access to a Windows domain environment, such information can assist with understanding the organization's Active Directory structure.

At this stage, however, the event did not explain **how the attacker obtained command execution**.

Therefore, the investigation moved backward through the process timeline.

7. Finding 2 — External Webhook Infrastructure

Earlier DNS telemetry revealed a query for:

`webhook.site`

The observed resolved address was:

`46.4.105.116`

The investigation also identified the following webhook URL in a decoded PowerShell payload:

`https://webhook.site/33c1dca9-b63a-470d-8d6d-cb46b76932ae`

The DNS event itself only establishes that the host resolved the external domain.

To determine why the server contacted the service, the investigation continued by examining PowerShell execution around the same period.

This correlation was important because a DNS query alone does not establish what information was transmitted.

The subsequent PowerShell analysis provided that missing context.

8. Finding 3 — Encoded PowerShell and Domain Enumeration

A Sysmon Event ID 1 showed execution of:

C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

The command line used the encoded-command option:

powershell -ec [Base64 encoded data]

The process was running from:

C:\Fusion21\cfusion\bin\

under:

NT AUTHORITY\SYSTEM

and its parent process was:

C:\Windows\System32\cmd.exe

Because the actual PowerShell command was encoded, the command was extracted and decoded using CyberChef.

The decoded PowerShell contained:

net user /domain

This command performs domain user enumeration.

The script then captured the command output and attempted to send it to:

<https://webhook.site/33c1dca9-b63a-470d-8d6d-cb46b76932ae>

The resulting activity can be summarized as:

```
PowerShell
↓
net user /domain
↓
Capture output
↓
Send output to webhook.site
```

This established a direct relationship between the external webhook infrastructure and the reconnaissance activity.

The server was therefore not merely communicating with webhook.site; the decoded command showed that information obtained from the compromised host was being sent to that service.

9. Finding 4 — Payload Decoding Using certutil.exe

The investigation then identified another important Sysmon Event ID 1.

The process was:

C:\Windows\System32\certutil.exe

with the command:

certutil -decode 123.tmp ..\wwwroot\cf_scripts\cfclient\huqVgdoFd.cfm

The current working directory was:

C:\Fusion21\cfusion\bin\

and the process was running as:

NT AUTHORITY\SYSTEM

The command shows that certutil.exe was being used to decode the contents of:

123.tmp

and write the decoded output to:

..\wwwroot\cf_scripts\cfclient\huqVgdoFd.cfm

Resolving the relative path gives:

C:\Fusion21\cfusion\wwwroot\cf_scripts\cfclient\huqVgdoFd.cfm

This was a major turning point in the investigation.

The .cfm extension and its location inside the ColdFusion web root suggested that the decoded payload was intended to be interpreted by the ColdFusion application.

The temporary file was then deleted as part of the command sequence.

The process chain can therefore be represented as:

Encoded Payload
↓
123.tmp

↓
certutil -decode
↓
huqVgdoFd.cfm

This provided evidence of deliberate payload deployment rather than ordinary ColdFusion application behavior.

10. Finding 5 — Analysis of the ColdFusion Web Shell

The newly created .cfm file was decoded using CyberChef.

The resulting content was written in **ColdFusion Markup Language (CFML)**.

The script contained logic that selected the command interpreter depending on the operating system.

For Windows, the relevant values were:

C:\Windows\System32\cmd.exe

and:

/c

The most important portion of the script was:

```
<cfexecute name="#p#" arguments='#a# "#c#"' outputfile="#GetTempDirectory()#f"
errorFile="#GetTempDirectory()#f" timeout=1></cfexecute>
```

This is the execution mechanism used by the web shell.

The script also checked for a parameter named:

c

The value supplied through this parameter was passed to the command interpreter.

Conceptually:

HTTP Request
↓
FORM.c
↓
cfexecute
↓
cmd.exe /c <command>
↓

Windows command execution

The script then wrote command output to a temporary file, read the result, returned the output, and deleted the temporary file.

This provided the attacker with a basic web-based command execution interface.

The discovery of this file explains how commands were being executed within the ColdFusion environment.

11. Finding 6 — PowerShell Reverse Shell

Further investigation revealed another encoded PowerShell payload.

After decoding the payload, the script was found to construct a TCP connection.

The destination was:

185.100.233.201

and the destination port was:

80

Therefore:

185.100.233.201:80

was identified as the reverse-shell destination.

The PowerShell script used a TCP client to establish communication with the remote system.

The basic flow was:

```
PowerShell
↓
Create TCP Client
↓
185.100.233.201:80
↓
Receive commands
↓
Execute commands
↓
Return output
```

This is distinct from the earlier webhook.site activity.

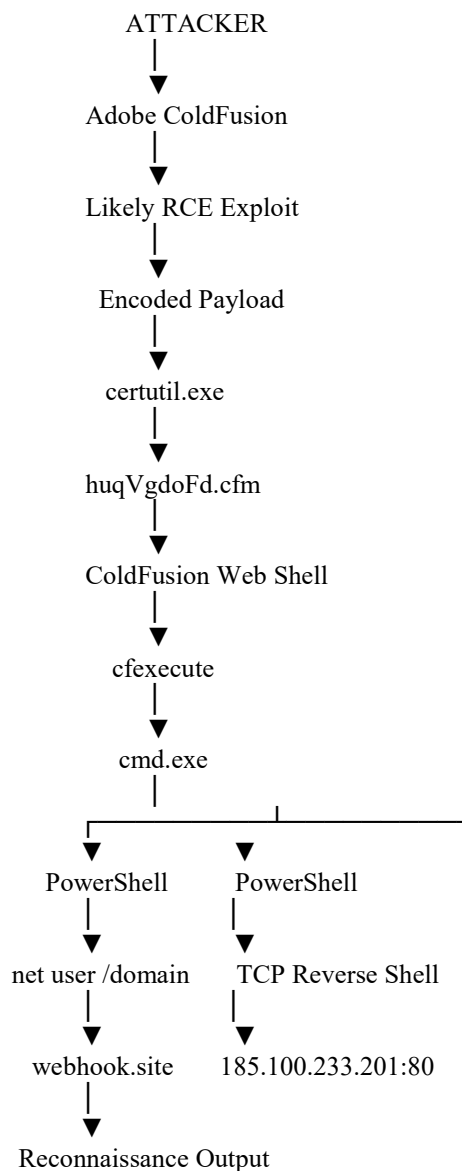
The two external destinations had different roles in the observed activity:

Destination	Observed Purpose
46.4.105.116 / webhook.site	Receiving reconnaissance output
185.100.233.201:80	Reverse-shell communication

Keeping these destinations separate is important when documenting the incident and creating indicators of compromise.

12. Attack Chain Reconstruction

At this point, the individual findings can be combined into a single attack sequence.



The later execution of:

```
nltest.exe /domain_trusts
```

provided additional evidence of domain reconnaissance.

The overall sequence demonstrates how the attacker moved from application-level access to operating-system command execution and then to reconnaissance and remote communication.

13. Indicators of Compromise

Host Indicators

```
C:\Fusion21\cfusion\bin\  
C:\Fusion21\cfusion\wwwroot\cf_scripts\cfclient\huqVgdoFd.cfm  
123.tmp
```

Processes

```
certutil.exe  
powershell.exe  
cmd.exe  
net.exe  
nltest.exe
```

Commands

```
certutil -decode  
net user /domain  
nltest /domain_trusts
```

Network Indicators

```
webhook.site  
46.4.105.116  
185.100.233.201:80
```

Web Shell

```
huqVgdoFd.cfm
```

These indicators should be used with contextual detection rather than treated as automatically malicious in isolation. Most of the executables identified are legitimate Windows binaries.

15. Conclusion

The investigation established a coherent sequence of malicious activity involving an Adobe ColdFusion server.

The investigation began with suspicious execution of:

```
nltest.exe /domain_trusts
```

Further examination revealed that the activity originated from the ColdFusion environment and was executed through PowerShell under the SYSTEM account.

Earlier telemetry showed the use of certutil.exe to decode an encoded payload into:

```
huqVgdoFd.cfm
```

Analysis of the decoded file revealed a ColdFusion web shell containing a cfexecute statement capable of executing operating-system commands.

The attacker then used PowerShell to perform domain reconnaissance through:

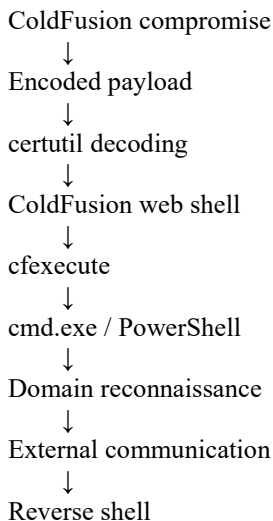
```
net user /domain
```

and transmitted the resulting information to an external webhook service.

Additional PowerShell activity established a TCP reverse shell toward:

```
185.100.233.201:80
```

The investigation therefore reconstructed the following attack progression:



The challenge identifies **CVE-2023-26360** as the likely vulnerability associated with the initial ColdFusion compromise.

Overall, the investigation demonstrates the importance of correlating **process creation, command lines, parent-child relationships, working directories, encoded payloads, DNS activity, and network destinations** rather than analyzing individual security events in isolation.